

Espacio Curricular: Programación Orientada a Objetos
 Curso: 5° año “B”
 Profesor: Armando B. VERA
 Tema: Conceptos básicos de C++

Objetivos

Se espera que los alumnos

- Escriban un primer programa en **C++** y utilicen los objetos **cout** de la clase **iostream** para salida por pantalla.
- Utilicen el compilador **g++** desde el CLI para generar archivos ejecutables.
- Utilicen el IDE **Geany** para escribir programas fuentes y compilen también desde el IDE.
- Utilicen y reconozcan los conceptos de **path** (relativo y absoluto) y configuren el IDE en función a las necesidades de su aplicación
- Al terminar este tutorial los alumnos deberán poder compilar un programa fuente escrito en **C++** tanto desde el terminal como desde el IDE.
- Escribir un programa en la cual puedan imprimir textos en pantalla.
- Reconocer las partes fundamentales de un programa en C++.
- Utilizar algunas sentencias del lenguaje como **main**, **cout**, **return** y **endl** y el operador de inserción en el flujo de salida **<<**.
- Reconocer y utilizar las directivas de preprocesador **#include** y **#define**

Estableciendo contacto

El arte de crear programas o **App** (Aplicaciones) involucran a muchas personas. Cada una se especializa en una fase y en conjunto obtienen la solución a un problema, o bien, brinda un servicio que facilita el desarrollo de una determinada actividad. Para el desarrollo de las aplicaciones se utilizan distintos **lenguajes de programación** que nos permiten escribir las instrucciones para que las computadoras puedan interpretar las acciones solicitadas por el ser humano. Nosotros vamos a utilizar, como ya mencionamos en otro apartado, el lenguaje de programación **C++** y como IDE vamos a utilizar **Geany** (aunque podemos utilizar cualquier editor y compilar directamente).

Ejemplo 1

Escriba un programa que muestre su nombre, fecha de nacimiento y deporte favorito. Cada dato en una nueva línea. Compile y ejecute el programa.

Para realizar esta actividad lo único que necesitamos es decirle al computador que muestre ciertos datos en el monitor o pantalla, a la que generalmente llamamos “dispositivo de salida”.

Para ello cargamos nuestro editor¹ (en nuestro caso Geany) en memoria y lo ponemos en primer plano. Para ellos hacemos **click** en **Aplicaciones Programación Geany**.

Para nuestros primeros trabajos vamos a crear los siguientes directorios o carpetas. En **Escritorio** vamos a crear el directorio **2024** y dentro de este **P00**. Por último, dentro de **P00** creamos el directorio **CPP**. En esta última carpeta vamos a guardar los archivos fuentes que vayamos creando. Eso nos permitirá contar con cierta organización de modo tal que todos tengamos una forma común de referirnos a estos *lugares*.

Una vez cargado el editor vamos a guardar el archivo nuevo con el nombre **misDatos.cpp**. Los archivos fuentes en C++ utilizan ésta extensión. Muchos compiladores utilizan esta información para la toma de decisiones.

¹En lenguaje corriente decimos que abrimos una ventana si utilizamos una interfaz gráfica

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      cout << "Nombre: Fulano " << endl;
6      cout << "Fecha de Nacimiento: 16/05/1998 " << endl;
7      cout << "Deporte favorito: Ajedres" << endl;
8      return 0;
9  }

```

La primera línea de este programa

```
1  #include <iostream>
```

utiliza un comando del **preprocesador**, una etapa previa del proceso de compilación propiamente dicho. Este comando sustituye en el archivo fuente dicho comando por la librería que representa. La librería **iostream** (más adelante veremos que es una clase) es un conjunto de código que permite ingresar datos a nuestro programa desde el teclado (o cualquier otro dispositivo de entrada) y también enviar datos del programa a cualquier dispositivo de salida como por ejemplo la pantalla. Normalmente nuestro sistema operativo tiene configurado como dispositivo de entrada por defecto al **teclado** y como dispositivo de salida por defecto al **monitor**. La siguiente línea

```
1  using namespace std;
```

es una directiva que indica que se ha de usar el **espacio de nombre estandar**. El lenguaje C++ soluciona definiendo espacios de nombres al gran problema que tienen los programadores cuando una aplicación comienza a quedarse sin nombres para la gran cantidad de identificadores que necesita. Definiendo espacios de nombres se pueden utilizar dos identificadores con el mismo nombre que hagan cosas “distintas”².

```
1  int main() {
```

Todo programa “ejecutable” en C++ tiene exactamente una función **main()** o función principal. Una función tiene por objeto realizar una tarea específica. Retorna un valor de un determinado tipo, en el caso de la función retorna un valor entero (por eso antes del main figura int). El int es un tipo de dato entero. Además tiene un nombre e inmediatamente un par de paréntesis dentro de los cuáles se escriben los argumentos (materia prima con la cual va a trabajar la función). En este caso está vacía, pero no siempre es así. Luego aparece una llave que da inicio a un bloque de instrucciones.

```

1      cout << "Nombre-----: Fulano " << endl;
2      cout << "Fecha de Nacimiento: 16/05/1998 " << endl;
3      cout << "Deporte favorito---: Ajedres" << endl;

```

Las tres instrucciones o sentencias utilizan el objeto **cout** (console output) de la clase **iostream** para enviar una cadena de texto a la consola. La cadena de texto es la que está entre comillas. En realidad al objeto **cout** hay que pensarlo como un puerta a un canal, dicho canal se cierra con el objeto **endl**. Todo lo que pongamos entre **cout** y **endl** es enviado al dispositivo de salida. El símbolo **<<** que siguen a **cout** y preceden a **endl** se conoce como **operador de inserción en el flujo de salida**.

Luego de las tres instrucciones aparece la sentencia

```
1  return 0;
```

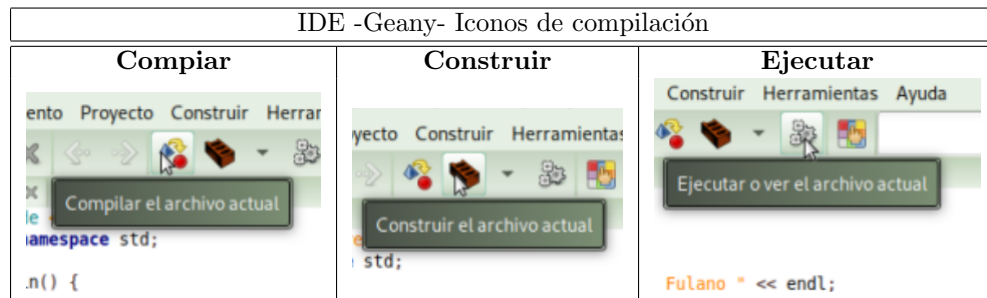
Esta instrucción hace que en el lugar donde se invoca a la función **main** se escriba un 0. El sistema operativo lee el contenido de esa posición de memoria y lo interpreta de acuerdo a ciertos criterios. Si

²Si bien esto suena un poco raro en principio pronto nos daremos cuenta que eso es plenamente posible. Por ejemplo abrir una puerta no es lo mismo que abrir un archivo. Una librería para un videojuego podría referirse a una puerta de un edificio, mientras que otra librería de tratamiento de archivos podría utilizar el mismo término en una función para abrir o cargar un archivo y ambas funciones podrían estar presente en la misma aplicación

retorna 0, como en este caso, significa que la función (en este caso, programa) se ejecutó con éxito. Un valor distinto de cero se interpreta como que ha ocurrido un error y el sistema operativo puede tomar distintas acciones a partir de ello. Después del `return` figura la llave que cierra el bloque de códigos abierto anteriormente. Debe quedar claro que cada llave de apertura debe tener su correspondiente llave de cierre. Ahí termina nuestro programa.

Compilando desde Geany

Para obtener un archivo ejecutable podemos hacerlo directamente desde el teclado con la tecla **F9** o bien con el mouse presionado el ícono de un ladrillo. Si todo va bien, aparecerá un mensaje informándonos que la compilación a terminado con éxito. De lo contrario, se mostrará uno o varios avisos de error de compilación. Para ejecutar el programa simplemente presionamos **F5** o con el mouse en el ícono que tiene los engranajes.



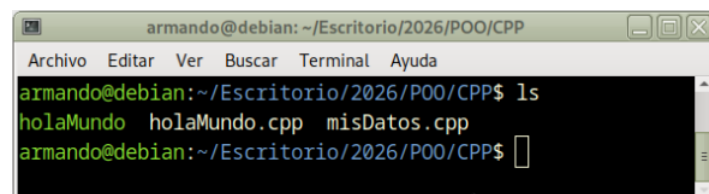
Compilando desde el Terminal

Abrimos el terminal y con las siguientes intrucciones cambiamos nuestro directorio actual.

```
~$ cd Escritorio/2026/P00/CPP
```

Una vez allí escribimos la siguiente instrucción:

```
$ ls
```

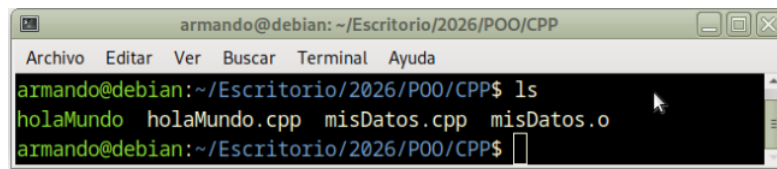


Podemos ver el ejecutable `holaMundo`, el archivo fuente `holaMundo.cpp` y el archivo fuente `misDatos.cpp`. Si quisieramos compilar el archivo pero no enlazarlo utilizamos la siguiente orden

```
$ g++ -c misDatos.cpp
```

Se genera un nuevo archivo conocido como `archivo objeto`. Este archivo está en lenguaje máquina pero aún no es ejecutable.

Si listamos nuevamente el contenido de la carpeta `CPP` podemos ver el nuevo archivo con nombre `misDatos.o`.



```

armando@debian: ~/Escritorio/2026/POO/CPP
Archivo Editar Ver Buscar Terminal Ayuda
armando@debian:~/Escritorio/2026/POO/CPP$ ls
holaMundo holaMundo.cpp misDatos.cpp misDatos.o
armando@debian:~/Escritorio/2026/POO/CPP$

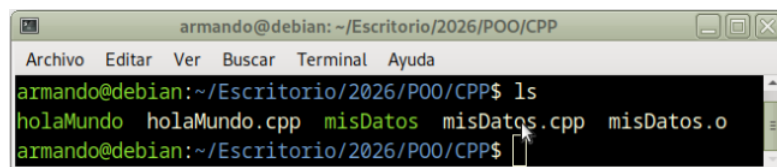
```

Para obtener un archivo ejecutable, es decir, para construir un archivo en programa utilizamos la siguiente orden

```
$ g++ -o misDatos misDatos.cpp
```

Ahora, listamos el contenido de la carpeta y podemos ver el archivo ejecutable **misDatos** en carpeta.

```
$ ls
```



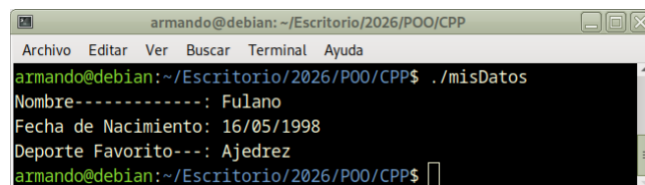
```

armando@debian: ~/Escritorio/2026/POO/CPP
Archivo Editar Ver Buscar Terminal Ayuda
armando@debian:~/Escritorio/2026/POO/CPP$ ls
holaMundo holaMundo.cpp misDatos misDatos.cpp misDatos.o
armando@debian:~/Escritorio/2026/POO/CPP$

```

Si quisieramos ejecutar desde el terminal lo hacemos con la siguiente orden

```
$ ./misDatos
```



```

armando@debian: ~/Escritorio/2026/POO/CPP
Archivo Editar Ver Buscar Terminal Ayuda
armando@debian:~/Escritorio/2026/POO/CPP$ ./misDatos
Nombre-----: Fulano
Fecha de Nacimiento: 16/05/1998
Deporte Favorito---: Ajedrez
armando@debian:~/Escritorio/2026/POO/CPP$

```

Hemos visto diferentes formas de obtener un archivo ejecutables a partir de un archivo fuente. El archivo fuente lo podemos escribir con cualquier editor de texto y si conoces las instrucciones del compilador no tienes por qué estar dependiendo de un IDE específico. Si bien la curva de aprendizaje es mayor, la solidez de los conocimientos adquiridos compensa ese mayor tiempo. Nosotros, como ya lo hemos mencionado vamos a trabajar con **Geany** por ser liviano y **sencillo** y, por sobre todas las cosas, de código abierto.

Ten en cuenta que

- En las Net del carrito, el path relativo a los archivos fuentes es: `Escritorio/2026/POO/CPP`
- Y el **path absoluto** es: `/home/alumno/Escritorio/2026/POO/CPP`
- El comando de compilación desde el terminal: `$ g++ -Wall -o holaMundo holaMundo.cpp`. La opción **-Wall** se incluye para que muestre todos los errores posibles y avisos.

Si tus archivos fuentes están en otro lugar tendrás que cambiar el path de tu entorno de desarrollo.